FinOps Multi-Agent Swarm Optimizer

Cloud Infrastructure Cost & Security Automation

Competition : Agentic AI World Cup — AgentSwarms School of Agentic AI
Submitted by: G S Shrikar
Email : shrikargs7@gmail.com
Platform : AgentSwarms (agentswarms.fyi)
Date : June 2025
Project : Cloud Infrastructure Optimizer & FinOps Swarm

Executive Summary

Cloud infrastructure waste is a silent tax on engineering teams. Over-provisioned resources accumulate unnoticed across deployments, orphaned storage volumes run indefinitely, and bloated Lambda packages introduce cold-start latency that degrades user experience — all while security misconfigurations lurk beneath review cycles too slow to catch them.

The FinOps Multi-Agent Swarm Optimizer is a four-node agentic pipeline built on the AgentSwarms platform that automates the full cost-security remediation cycle: from raw Terraform and serverless config ingestion, through automated waste quantification, corrected configuration generation, human review, and a final compliance sweep — end to end, in a single swarm run.

On a realistic demo configuration, the swarm identified $430–$610/month in recoverable cloud waste and produced copy-pasteable Terraform fix blocks, all cleared by the Compliance Officer in one automated security pass.

$430–$610	4 Nodes	1 Run
Estimated monthly savings	Swarm architecture	Audit → fix → cleared

Why Agentic AI?

A traditional single-agent approach struggles because cloud optimisation requires multiple specialised perspectives.

The FinOps analyst focuses on cost efficiency.

The Infrastructure Engineer focuses on implementation.

The Security Officer focuses on compliance.

Combining these responsibilities inside one prompt often leads to conflicting objectives and reduced reliability.

By distributing responsibilities across specialised agents, each agent operates within a focused context window and produces more consistent outputs.

2.Problem Statement

Modern cloud deployments face three compounding failure modes that no single engineer or tool addresses in combination:

- Over-provisioned resources: Lambda functions allocated 3 GB of memory handling sub-100 ms workloads; EC2 instances idling at 2–5% CPU utilisation on premium instance types; EBS volumes detached months ago still accruing charges.
- Serverless cold-start latency: Bloated deployment packages (100 MB+) force runtimes to initialise slowly, degrading p99 latency for end users without ever appearing in cost dashboards.
- Security drift: Infrastructure as Code reviews happen infrequently. Wildcard IAM permissions, publicly accessible S3 buckets, and missing encryption parameters slip through and accumulate over time.

The root problem is that fixing these issues requires three distinct skill sets — FinOps analysis, DevOps engineering, and security compliance — applied in sequence. A swarm of specialised agents, each owning one domain, with a human checkpoint before code transitions forward, is the correct architectural response.

3.Swarm Architecture Overview

The pipeline runs as a linear five-step chain on the AgentSwarms platform. Each node receives the output of the previous node as its input context, ensuring full information continuity across the pipeline.

Node	Name	Role	Model tier	Temperature
1	Cost & Latency Auditor	Scan & quantify waste	Fast / reasoning	0.1
2	Infrastructure Engineer	Generate fix config	Strong coding	0.3
HITL	Approvals Gate	Human review checkpoint	— (platform node)	—
3	Compliance & Security Officer	Security sweep & final report	Standard instruction	0.0

4. Node-by-Node Specification

4.1 Node 1 — Input Trigger

Node type: Input (Flow node)

Platform node: Input — Seed value for the run

Function: Accepts raw user text: a Terraform config, serverless YAML, or cloud log snippet. This is the swarm's entry point — no model is invoked here, it simply seeds the pipeline with the raw infrastructure content to be analysed.

During the demo run, the following mock Terraform configuration was pasted as input:

```
resource "aws_lambda_function" "api_handler" {
  function_name = "order-processor"
  runtime       = "nodejs18.x"
  memory_size   = 3008          # Over-provisioned
  timeout       = 900
  filename      = "dist/build.zip"  # 187 MB — bloated package
}

resource "aws_instance" "backend_server" {
  instance_type = "m5.4xlarge"
  tags = { avg_cpu_util = "3%" }  # Severely under-utilised
}

resource "aws_ebs_volume" "legacy_data" {
  size = 500
  tags = { Attached = "false", LastUsed = "2023-11-01" }  # Orphaned
}

resource "aws_s3_bucket" "asset_store" {
  bucket = "prod-asset-store-v2"  # No lifecycle policy
}
```

4.2 Node 2 — Cost & Latency Auditor

Node type: Agent (LLM call)
Model: GPT-4o-mini or Claude 3.5 Haiku (fast, reasoning-focused)
Temperature: 0.1 — Near-deterministic analytical output
Tools enabled: Calculator (platform built-in) for precise waste quantification

System Prompt

```
You are an expert FinOps Data Analyst specialising in cloud cost optimisation and resource allocation.
Your job is to analyse the raw cloud logs or infrastructure configurations provided by the user.

Look specifically for:
  1. Over-provisioned resources (full servers running when serverless functions or smaller
     instances could handle the load).
  2. Serverless cold-start latency signals or bloated deployment packages.
  3. Unused or orphaned storage volumes.

Output your findings as a clean, structured Markdown table detailing: Resource Name,
Detected Issue, Cost Impact (High/Medium/Low), and a brief recommendation.
Do not write code; focus purely on the analysis.
```

Design rationale

Temperature 0.1 is chosen deliberately. Cost analysis must be reproducible — the same input config should always surface the same issues. Any creative variance here introduces audit inconsistency. The Calculator tool allows exact monthly cost projections based on AWS pricing rather than qualitative estimates.

The agent is instructed not to write code. This strict separation of concerns prevents the auditor from making assumptions about fix strategies — that responsibility belongs entirely to the Infrastructure Engineer in the next node.

Sample output — Findings table

Resource	Detected issue	Impact	Recommendation
aws_lambda_function.api_handler	3008 MB memory; 187 MB package causes cold-start latency	High	Reduce to 512–1024 MB; tree-shake deps; enable SnapStart
aws_instance.backend_server	m5.4xlarge at 3% avg CPU — severely over-provisioned	High	Downsize to t3.medium or migrate to Lambda/Fargate
aws_ebs_volume.legacy_data	Orphaned 500 GB gp2 — unattached since Nov 2023	Medium	Snapshot and delete; use gp3 if reattached (20% cheaper)
aws_s3_bucket.asset_store	No lifecycle policy — uncontrolled storage accumulation	Low	Add Intelligent-Tiering rule; set version expiry

4.3 Node 3 — Infrastructure Engineer

Node type: Agent (LLM call)
Model: GPT-4o or Claude 3.5 Sonnet (strong coding model)
Temperature: 0.3 — Structured, accurate code generation with slight flexibility
Tools enabled: None — pure code generation from audit context
Input: Full findings table from Node 2 (Cost & Latency Auditor)

System Prompt

```
You are an expert DevOps and Cloud Infrastructure Engineer. You will receive an analysis
report from the Cost & Latency Auditor agent.

Your job is to generate concrete configuration solutions to fix the identified issues.
- If serverless cold starts are a problem, provide specific memory tuning configs or
  provisioned concurrency blocks.
- If resources are over-provisioned, write the corrected Terraform or serverless
  configuration blocks.

Format your entire response using Markdown. Separate your architectural recommendations
from the raw, copy-pasteable configuration code blocks.
```

Design rationale

Temperature 0.3 (versus 0.1 for the auditor) reflects the nature of this task. Code generation benefits from a small degree of reasoning flexibility — there are often multiple valid ways to right-size a resource, and the engineer should be able to choose an idiomatic approach rather than producing a rigid mechanical output. However, it stays well below 0.5 to prevent hallucinated API fields or incorrect configuration syntax.

The agent receives the full audit table as its context. This is intentional — the engineer needs the complete picture of what was flagged and why, not just a list of resource names, to make sound architectural recommendations.

Sample output — Corrected Terraform blocks

```
# Lambda — right-sized memory + provisioned concurrency
resource "aws_lambda_function" "api_handler" {
  function_name = "order-processor"
  runtime       = "nodejs18.x"
  memory_size   = 512          # was 3008 MB
  timeout       = 30           # right-sized for actual p99 latency
  filename      = "dist/build.zip"
}

resource "aws_lambda_provisioned_concurrency_config" "api_handler_pc" {
  function_name              = aws_lambda_function.api_handler.function_name
  qualifier                  = aws_lambda_alias.live.name
  provisioned_concurrent_executions = 2
}

# EC2 — downsize to burstable tier
resource "aws_instance" "backend_server" {
  instance_type = "t3.medium"  # was m5.4xlarge (~$0.77/hr -> ~$0.04/hr)
}

# EBS — snapshot before decommission
resource "aws_ebs_snapshot" "legacy_backup" {
  volume_id   = aws_ebs_volume.legacy_data.id
  description = "Final snapshot before decommission"
}

# S3 — lifecycle policy
resource "aws_s3_bucket_lifecycle_configuration" "asset_store_lc" {
  bucket = aws_s3_bucket.asset_store.id
  rule {
    id     = "intelligent-tiering"
    status = "Enabled"
    transition {
      days          = 30
      storage_class = "INTELLIGENT_TIERING"
    }
    noncurrent_version_expiration { noncurrent_days = 90 }
  }
}
```

4.4 Human-in-the-Loop Gate (Approvals Node)

Node type: Approval — Pause for human (Flow node)

Platform node: Approval (AgentSwarms built-in HITL node)

Model: None — no LLM invoked

Function: Pauses swarm execution and surfaces a review notification in the AgentSwarms UI. The reviewer reads the Infrastructure Engineer's generated configuration, approves or rejects, and the swarm either continues to the Compliance Officer or terminates.

Design rationale

This is the most deliberately placed node in the architecture. Agentic systems that autonomously generate and forward infrastructure configuration code without a human checkpoint are an operational liability — an incorrectly scoped security group or a resource deletion without a snapshot backup can cause production incidents.

The HITL gate ensures that a human engineer reviews the generated Terraform blocks before they are passed to the Compliance Officer for final sign-off. This aligns with responsible agentic AI design: the swarm handles the analytical and generative workload, but a human retains decision authority over what actually ships.

During the video demo run, the gate visibly paused execution mid-stream, displaying the Infrastructure Engineer's output for review. After approval, the swarm resumed automatically.

4.5 Node 4 — Compliance & Security Officer

Node type: Agent (LLM call)

Model: Standard instruction-following model (GPT-4o or equivalent)

Temperature: 0.0 — Absolute consistency required for compliance decisions

Tools enabled: None — deterministic rule-based evaluation

Input: Approved configuration blocks from the Infrastructure Engineer (post-HITL)

System Prompt

```
You are a strict Cloud Security and Compliance Officer. You will receive a proposed
infrastructure change from the Infrastructure Engineer.

Review the code blocks and recommendations to ensure they do not introduce security risks.
Check for:
  1. Publicly accessible cloud resources (e.g., wide-open ingress rules, public S3 buckets).
  2. Over-privileged roles (e.g., wildcards like '*' in IAM permissions).
  3. Missing encryption parameters.

If the code looks safe, prefix your response with '     [SECURITY CLEARED]' and compile
the final, polished FinOps optimisation report. If an issue is found, flag it clearly.
```

Design rationale

Temperature 0.0 is non-negotiable here. Security compliance decisions must be identical across every run against the same input. Any stochasticity in a compliance check creates

exploitable inconsistency. The agent uses rule-based deterministic reasoning: if a wildcard exists in IAM, flag it — no probability involved.

The prefixed output format ([SECURITY CLEARED] or explicit flags) is a deliberate interface design. It allows downstream automation — or a human reading the final report — to immediately parse the compliance verdict without reading the full analysis.

Sample output — Security clearance report

Security check	Result	Note
Public resource exposure	Pass	No public-facing ingress rules or public S3 ACLs introduced
IAM privilege scope	Pass	No wildcard (*) IAM actions or resources added
Encryption at rest	Advisory	Add explicit encrypted = true to EBS snapshot resource for auditability
EBS orphan deletion	Pass	Snapshot created before volume decommission — data retention satisfied
Provisioned concurrency scope	Pass	Set to 2 — within safe cost bounds, no runaway scaling risk

5. Projected Cost Savings

The following estimates are based on AWS on-demand pricing (us-east-1) applied to the demo configuration. The Calculator tool in Node 2 was used to derive these figures.

Resource	Current monthly cost	Optimised cost	Monthly saving
EC2 m5.4xlarge → t3.medium	~$556/mo	~$29/mo	~$527/mo
Lambda 3008 MB → 512 MB	~$24/mo (est.)	~$4/mo (est.)	~$20/mo
EBS gp2 500 GB orphan deleted	~$50/mo	$0	~$50/mo
S3 Intelligent-Tiering (est. 30% reduction)	Variable	~30% less	~$10–40/mo
Total estimated monthly saving			**$597–$637/mo**

6. Data Flow & Agent Communication

Each node passes its complete output to the next node's input context. No summarisation occurs between nodes — the full text is forwarded to preserve analytical fidelity. This is intentional: the Infrastructure Engineer must see the complete audit table, not a truncated summary.

From		To		Data passed
Input Trigger (Node 1)	→	Cost & Latency Auditor (Node 2)		Raw Terraform / config text
Cost & Latency Auditor (Node 2)	→	Infrastructure Engineer (Node 3)		Markdown findings table with 4 flagged resources
Infrastructure Engineer (Node 3)	→	HITL Approvals Gate		Generated Terraform fix blocks + architectural notes
HITL Gate (approved)	→	Compliance Officer (Node 4)		Approved config blocks
Compliance Officer (Node 4)	→	Output		[SECURITY CLEARED] + final FinOps report

7. Key Design Decisions

Strict role separation

Each agent is prohibited from performing the work of the other agents. The auditor produces no code. The engineer performs no security checks. The compliance officer generates no configuration. This is deliberate — it prevents scope creep within a single agent context window, which degrades output quality in multi-task prompts, and makes the swarm's outputs independently auditable.

Temperature ladder: 0.1 → 0.3 → 0.0

Temperature assignments follow the nature of each task. Analysis (Node 2) is set to 0.1 — near-deterministic, reproducible findings. Code generation (Node 3) is 0.3 — slight variance allows idiomatic choices without hallucinating incorrect API fields. Compliance (Node 4) is 0.0 — security decisions must be identical for identical inputs. No arbitrariness in security checking is acceptable.

HITL gate placement

The human review gate is placed between the Infrastructure Engineer and the Compliance Officer — not at the very end. This means a human reviews the proposed code before it is security-cleared, not after. The compliance check then validates what a human has already approved, creating a dual-layer verification: human intent check followed by automated rule check.

Calculator tool scoped to the auditor only

The Calculator tool is enabled only on Node 2. Neither the engineer nor the compliance officer require computational tools — the engineer works from structured text output, and the compliance officer applies rule-based logic. Limiting tool access to where it is actually needed reduces noise and potential for misuse in other nodes.

8. Conclusion

The FinOps Multi-Agent Swarm Optimizer demonstrates that agentic AI is not just a research concept — it is a practical engineering tool for the problems that slow teams down most: the slow, multi-disciplinary, sequential work of cloud cost review and security compliance.

By decomposing the workflow into specialist agents with distinct temperature profiles, clear input/output contracts, and a mandatory human checkpoint, the swarm achieves something no single-agent prompt can: reliable, repeatable, auditable infrastructure remediation at the speed of an LLM.

The architecture is designed to extend naturally — a fifth node for cost forecasting, a router agent to handle multi-cloud providers, or a loop node for iterative compliance hardening. The foundation is built for it.

9.Appendix - Agent Configuration Reference

Node	Agent name	Model tier	Temp	Tools	Output format
1	Input Trigger	— (Flow)	—	—	Raw text passthrough
2	Cost & Latency Auditor	Fast / reasoning	0.1	Calculator	Markdown table
3	Infrastructure Engineer	Strong coding	0.3	None	Markdown + code blocks
HITL	Approvals Gate	— (Flow)	—	—	Human decision (approve/reject)
4	Compliance & Security Officer	Standard instruction	0.0	None	/ + final report

Screenshots

Overall agent

Seeks approval from Human